Streaming Analytics

Hello and welcome to the module on Streaming Analytics.

Proprietary + Confidential

Learn how to ...

Describe how to implement a streaming analytics solution using Pub/Sub, Dataflow, and BigQuery

- Identify the components of a streaming analytics solution on Google Cloud
- Align Google streaming analytics solutions with Kafka and AWS Kinesis
- Explore design patterns and optimization considerations for streaming analytics solutions
- Create a streaming pipeline using Pub/Sub and Kafka

Google Cloud

In this module, we will look at how to implement the streaming analytics solution using three products: Pub/Sub, Dataflow, and BigQuery.

- So those are the three components of the Streaming Analytics solution that we talk about,and why it is that these are the three components.
- And then we'll also look at how this compares with things like Kafka and AWS Kinesis and how you basically do migrations.
- Then we will look at design patterns and optimization considerations when you're building streaming analytics pipelines.
- And finally, we will create a streaming analytics pipeline using Pub/Sub and Kafka.

Proprietary + Confidential

Agenda

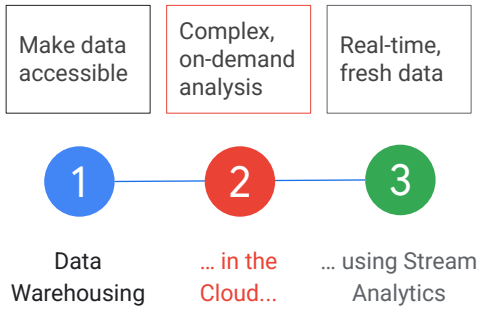
- Why Streaming Analytics?
- The Pub/Sub Service
- Lab: Streaming Kafka to Pub/Sub
- Dataflow Windows and Triggers
- Dataflow Sources and Sinks
- Migration and Adoption Challenges
- Optional Lab: Simulating a Data Warehouse in the Cloud Using BigQuery and Dataflow
- Summary

Google Cloud

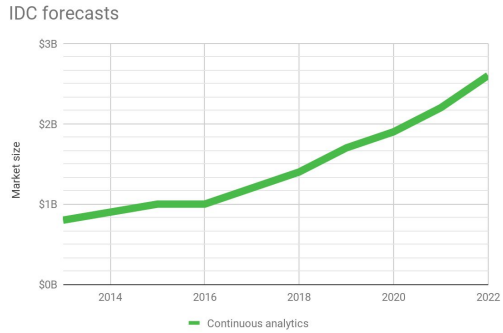
Let's start by talking about why you might want to do streaming analytics in the first place.

Proprietary + Confidential

Stream analytics: The next thing in big data



Evolution of big data in the cloud



Stream analytics: From $1.5B in 2019 to $3B in 2022

Google Cloud

Streaming analytics is in many ways the next big thing in big data. The first phase was all about breaking down data silos. It's about taking the data that's available throughout the organization, making it accessible to everybody. And that was the whole point of data warehousing. It's to make data accessible so that you can join different datasets, you can carry out queries between them without worrying about all of these datasets being in different formats, etc. That's what data warehousing was, and that's the first big evolution in the cloud of being able to do data warehousing - breaking down data silos and bringing data from across the organization.

But when you move into the cloud, the other thing that the cloud provides is that that analysis that you're doing can be ad hoc, can be on demand, can be quite complex. Later on in this course, we will look at how to do GIS queries and how to do machine learning. And all of these things are possible because you're doing data warehousing in the cloud. So that's a second big evolution of big data - being able to do ad hoc analysis over large datasets. So things that used to take hours can now take minutes. So that's the second big evolution.

But now, if the analysis that used to take hours takes minutes, then people start to ask, OK, how fresh is this analysis thing? If you're going to do an analysis and it takes 15 seconds, but the data that you've done the analysis on is yesterday's data, it starts to not be as appealing anymore. Well, if you can now do the analysis extremely fast, maybe the analysis that you should be doing is on fresh data, data that's real time that just arrived. And there's a lot of business impact that can be had if you're providing analysis on up to the minute data. And that's what streaming analytics is about. And

so that's where the next big thing, we will not be able to do data warehousing in the cloud if we didn’t do data warehousing in the first place. If we did not break down silos, we couldn't do the on-demand analysis and we couldn’t join large datasets. And if you are not able to do it very fast, no-one would be asking you for up to the minute analysis if that analysis takes hours. But now that it takes seconds, now you're basically getting asked, well, can it also reflect fresh and real time data. So stream analytics is this natural evolution, and as companies go through an evolution of becoming more and more mature in terms of the analytics capabilities that they can do, they go through this exact same evolution. They end up with the need for real time fresh data because real time up to the minute data is more valuable to them. Giving a coupon to a customer as they enter the store has a lot more value than shipping them a coupon hoping that they do come to the store later.

So when you look at the trend in the industry, streaming analytics in 2019 was about $1.5 billion, and is expected to double by 2022. So that's another way to measure how it's the next big thing when it comes to big data processing.

Proprietary + Confidential

Stream analytics has many applications

Data integration (10 sec - 10 min)	Online decisions (100 ms - 10 sec)
• Data warehouses becomes real-time • Take load off source databases with change data capture (CDC) • Microservices require databases and caches	• Real-time recommendations • Fraud detection • Gaming events • Finance back office apps

Google Cloud

So what are some of the applications of streaming analytics? Well, the first thing we've just talked about. Data warehouses themselves become real time and data integration, the data that's coming in, becomes more up to the minute. So now you can stop thinking about that data warehouse being periodically refreshed, or whenever there's a transaction that's happening you're immediately reflecting it in the data warehouse instead of capturing all the changes once a day and moving into the data warehouse, for example.

So this also helps you take the load off source databases because if you're doing change data capture once a day, the time at which you're doing the change data capture, you're doing a lot of querying on that source database, whereas if you're doing it as transactions are arriving, then you're basically allowing yourself to spread the load a bit.

The third reason why you may want to do streaming data analytics is because you have microservices and they all require databases and they require caches. These microservices also require that data be up to date, so you're looking at how to enable your data warehouse to reflect up to date data so that all these microservices and the caches that they're using are all correct.

The first reason for doing streaming analytics was all around IT. Because the timescales that we're talking about when we say a data warehouse becoming real time, when we're talking about just reflecting the data, preventing the change data capture from over-burdening the database, you're often talking about making your

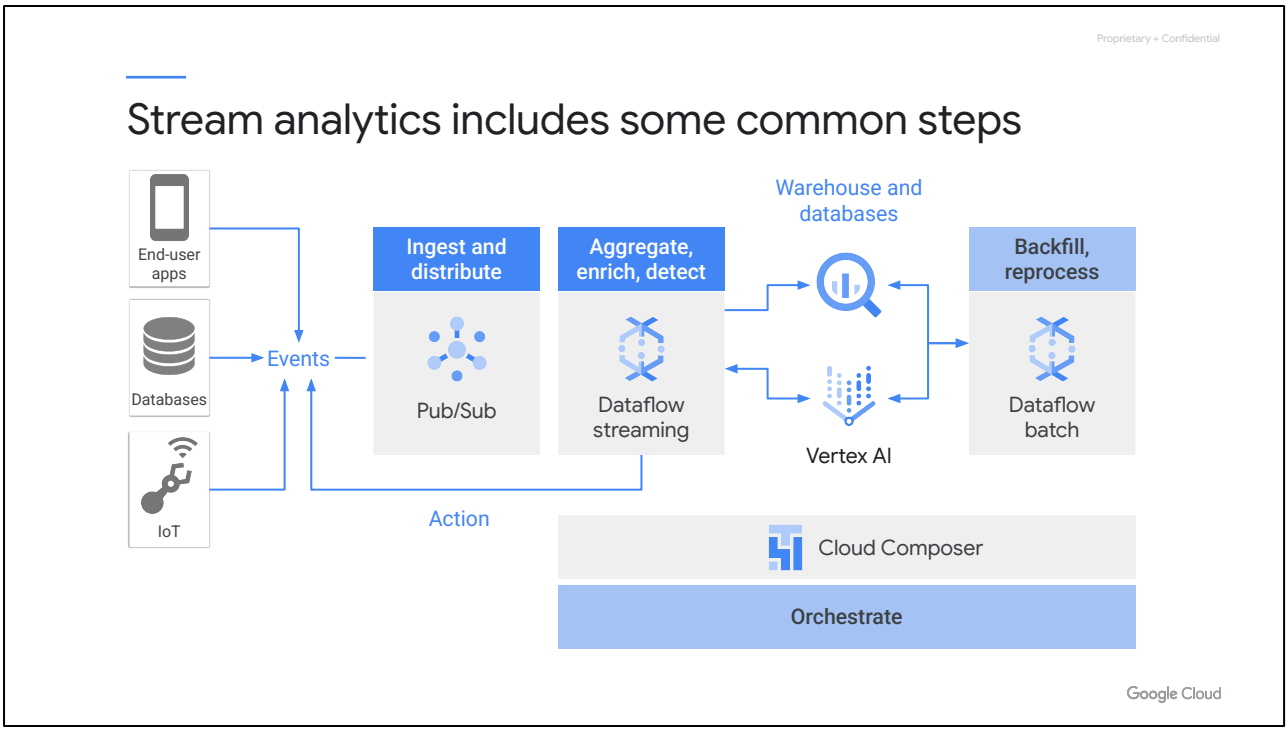
integrations and so on in the order of seconds to minutes.

But the second range of applications is all to do with online decisions, decisions that you want to make on the fresh data. For example, you want to do real-time recommendations. When somebody visits a web page, you want to show them the kinds of things that that you think they would be interested in. And that should be based on the search queries and things that they did on your website just a few seconds back. So when you think about online decisions, you're now talking about timescales in the order of milliseconds to maybe a few seconds.

So when you're talking about fraud detection, you want to detect fraud on the transaction that is actually happening right now. You don't want to approve the transaction and then 10 minutes later, or one day later, or one week later say, oh, my, that was fraud. That doesn't make any business sense. Fraud detection needs to be real time, it needs to be an online decision.

Same thing with gaming events. If someone is playing a game and you need to figure out what's the next challenge that you're going to give to them, or what's the next character that's going to happen, or what's the next in-app purchase that you're going to offer to them, that very much depends on how they're playing the game. So that's again, an online decision.

But this is not just about digital natives and so on. Even when you think about back office applications, if the decision needs to be made very fast, then you need online decisions. So a lot of financial back office applications also need online decision-making to happen, and at that point you're talking about streaming analytics.



So when you're doing streaming analytics, the typical architecture involves a few common steps. Firstly, you're going to have events. These events are going to drive a whole bunch of analytics processing to happen. The point about streaming analytics is that these events are continually happening.

Events could be user-driven events, maybe from mobile phones or web applications. There could be database-driven events, things like changes to a database or, backfills that need to happen, or errors that happen in your applications, etc., so what's called machine-driven events. The third kind of events are things that happen very routinely, maybe like Internet of Things, like sensor data, it's just collected on a schedule. It's just collected a hundred times a second and you're basically getting those events one at a time. So that's one thing. These are the things that we're processing - all of these events.

The next part of streaming analytics is what you're going to do if there is an event. So first of all, you need to get those events. So there is an ingest mechanism. These events, in general, don't happen in your data center. The database events might, but your end-user events, your sensors, they're not in your data center they're somewhere else. You need to get these events into your processing pipeline - that's the ingest. Ideally, your ingest mechanism is global in nature and that's what Pub/Sub is. We'll talk about Pub/Sub in more detail, but essentially it's the way in which we ingest these events no matter where they happen and no matter how often they happen. Every event that's in Pub/Sub then gets aggregated, enriched, detected, and processed using a pipeline tool called Dataflow.

So Dataflow is where we write the code to process these events. So the code that's processing these events might process them one at a time. More often it processes them in streaming windows. These windows could be time based. For example, it could be that I'm going to take all my sensor data and find the average value in the last five minutes. It could be processing events as they happen, for example I'm going to find the maximum temperature, or I'm going to find out every time somebody liked something on my app, or I'm going to find every change that happened to my database.

Or it could be event windows driven by some other thing. It could be based on a user session - every time I'm going to look at all the actions that somebody did in a session and figure out how many times did they try the different actions. And that could be the thing that the Dataflow pipeline does.

Proprietary + Confidential

Agenda

Why Streaming Analytics?

The Pub/Sub Service

Lab: Streaming Kafka to Pub/Sub

Dataflow Windows and Triggers

Dataflow Sources and Sinks

Migration and Adoption Challenges

Optional Lab: Simulating a Data Warehouse in the Cloud Using BigQuery and Dataflow

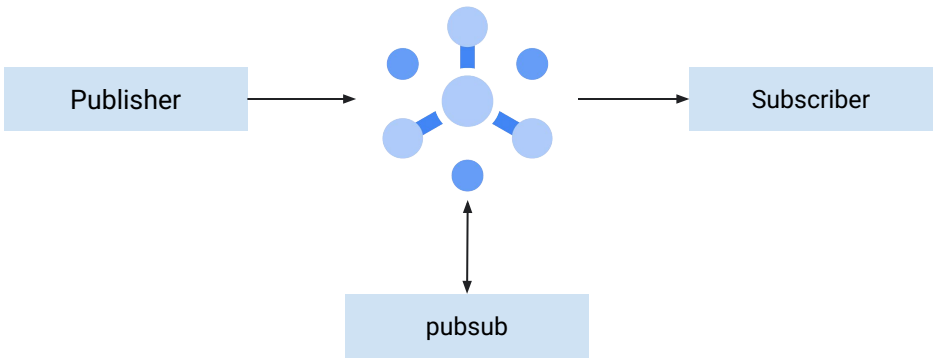
Summary

Google Cloud

When events come in, whether they come in from user-generated events, machine-generated events, or IoT sensors, they land up in Pub/Sub.

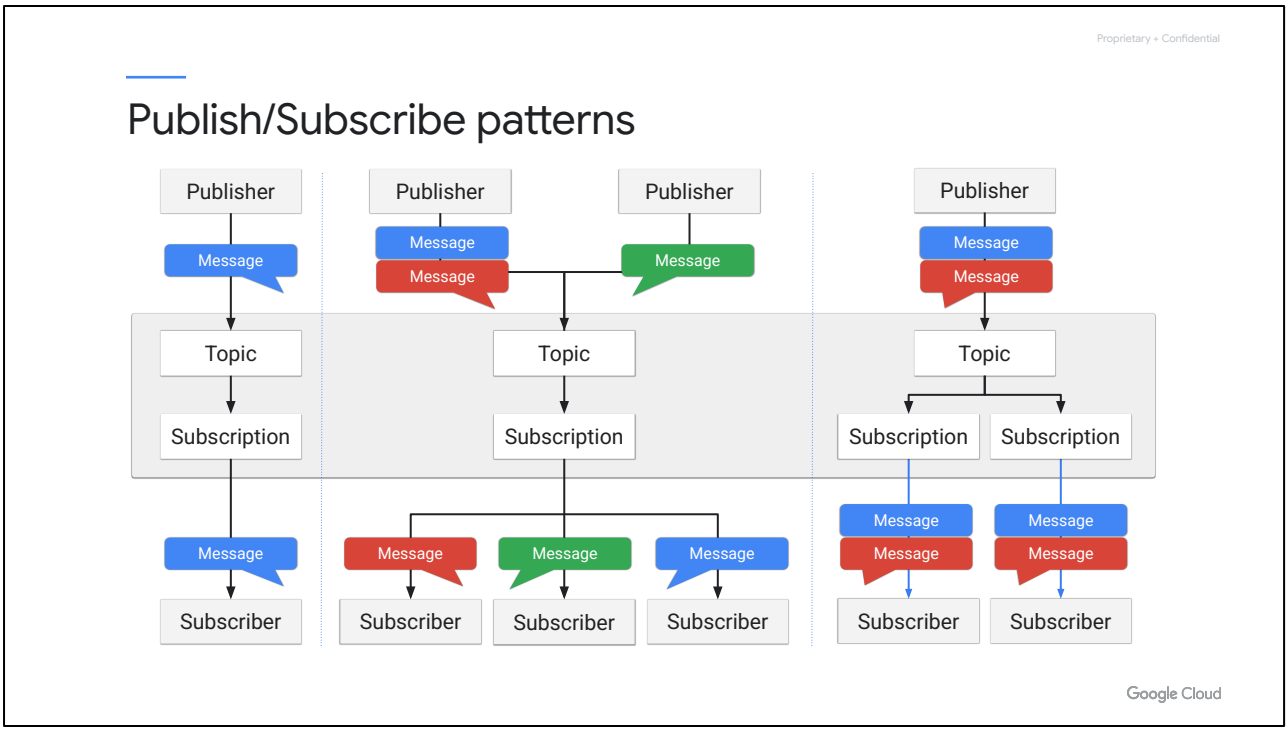
Proprietary + Confidential

The Pub/Sub service



Google Cloud

Pub/Sub is essentially a way by which you can have events getting published into Pub/Sub and you can have subscribers pulling from those events or being notified about those events. So it is a publish and subscribe service and that's what Pub/Sub is. So how do you use Pub/Sub? Let's look at a few patterns.



One option, and probably the one that you're thinking about, is a one-to-one, or a queue. In other words, a publisher publishes a message into a topic and a subscriber gets the message from the topic. The way Pub/Sub works though, is subscribers do not get messages directly from the topic. Instead, every subscriber creates a subscription and subscribers get their messages from the subscription. I'll talk about why that's a useful concept in a little bit, but essentially publishers publish messages and subscribers receive messages. This is a one-to-one relationship, it’s a queue, and it’s not very common.

Probably the way that you might want to think about this is that it's a many-to-many relationship. You might have multiple publishers, all of whom are publishing to the same topic. Maybe publisher number one has a couple of messages that they publish into the topic, and publisher number two has another message they’re publishing into the topic. All these messages are available in the topic and you can have a subscriber create a subscription. It could be that you have different subscribers creating different subscriptions or multiple subscribers sharing a subscription. But then, once you have a subscription, all the messages that go to that topic are available to all the subscribers on that subscription. In this case, these subscribers each receive a message. On the other hand, if subscribers one and two had one subscription, and subscriber three had a separate subscription, subscriber three will get all three messages and subscribers 1 and 2 will share the three messages between them. So there's many different ways that you could do it. In other words, Pub/Sub supports a fan-in mechanism - it supports a distribution mechanism. Because of this concept of a subscription, they can create a topic and a subscription itself can be shared, whereas

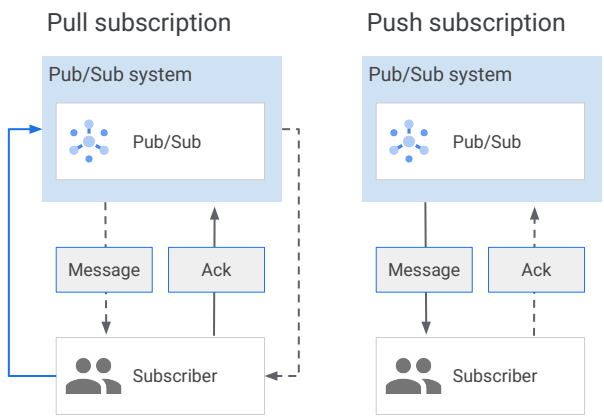
publishers publish it to a topic.

The most common situation, though, is a one-to-many. You have a publisher, someone who can create events - think of your apps, for example, publishing into a topic, and you can have multiple subscribers. Each of these is a different application that needs to do different things with those messages, each of which creates their own subscription and processes all of the messages that arrive in the subscription. So this is a one-to-many, or a fan out, and it's probably the most common.

Proprietary + Confidential

At least once delivery guarantee

- A subscriber ACKs each message for every subscription
- A message is resent if subscriber takes more than `ackDeadline` to respond
- A subscriber can extend the deadline per message



Google Cloud

With Pub/Sub you have an at least once delivery guarantee. So in other words, if the publisher publishes a message, you as a subscriber who has a subscription, you're guaranteed to get the message.

But it could be possible that you don't acknowledge the message. Maybe the subscriber crashes before it can acknowledge the message. And at that point, the publisher doesn't know if the subscriber crashed before it processed the message or after it processed the message. So if it doesn't receive an acknowledgement within a specific deadline, Pub/Sub resets the message. So it is possible that a subscriber might receive a message more than once if for whatever reason it doesn't acknowledge the message in time.

A subscriber can also say, well, I'm running late and I want to extend the deadline for the acknowledgement so I have more time to acknowledge it. But regardless, there's always a possibility that you didn't acknowledge and therefore you might get the message a second time.
Now, how do subscribers work? Subscribers set up a pull subscription or a push subscription.The pull and the push are from the perspective of Pub/Sub. If it's a pull subscription, then the subscriber is pulling it whenever it needs to. If it's a push subscription, it’s saying here's a callback, please call me whenever there's a new message. So both are available from client libraries.

Proprietary + Confidential

Latency, order, duplication will happen

- Latency -- no guarantees
- Messages can be delivered in any order, especially with large backlog
- Duplication will happen
- Duplication will happen

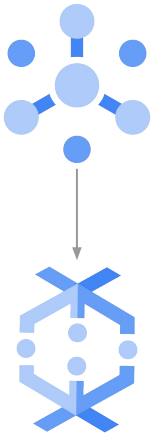
Google Cloud

- Pub/Sub is a global system into which anyone can publish, so there's going to be latency. There's no guarantee of how long it’s going to take because of the network system.
- The messages are not guaranteed to be in order. This is gonna give a lot of people heartburn. There is no ordering with messages. So when publishers publish messages, subscribers receive those messages, and they receive those messages at least once, but there is no ordering guaranteed if they're not in any kind of time order. Isn't that a problem? Not really. Okay. We'll talk about that.
- Another thing is that you should realize is that duplication will happen. Duplication will happen because there is no guarantee that when the subscriber acknowledges receipt, it actually arrives in time, which takes you back to the first point about latency. So latency, ordering, and duplication - those are things to keep in mind when we talk about Pub/Sub.

Proprietary + Confidential

Exactly once, ordered processing

- Pub/Sub delivers at least once
- Dataflow: Deduplicate, order, and window
- Separation of concerns -> scale



Google Cloud

Let's take a look at the ordering.

- You basically say that Pub/Sub delivers at least once, so it's guaranteed to deliver it to you. So if the subscriber, for whatever reason crashes, it doesn't acknowledge it, Pub/Sub will deliver it again. So it's trying to make sure that this message is delivered. It keeps messages around for seven days and will keep trying to get it back to you. So Pub/Sub delivers at least once.
- But because it delivers at least once, there's a possibility of duplication. So what should you do? You should process those messages that come into Pub/Sub with Dataflow. Dataflow will guarantee that you are processing it exactly once. Dataflow will deduplicate, it will order things, put them in a window, and process them. So in other words, Pub/Sub doesn't give you ordering guarantees and doesn't give you duplication guarantees, but guarantees delivery. The two missing guarantees are provided by Dataflow. This combination of Pub/Sub and Dataflow give you what you need. It gives you guaranteed delivery and it gives you exactly once, ordered processing. So don't look at the products separately, look at them together. As a solution, they actually solve what you need.
- And why do we do it that way? It's because by separating it out, these two concerns, it allows us to scale massively. We'll have a look at the tradeoffs that other messaging systems make. The big advantage of Pub/Sub is that it's global and it allows you to you get all of these things with the assumption that you're processing the messages that arrive in Pub/Sub through Dataflow.

Proprietary + Confidential

Pub/Sub API features

- Push, pull, streaming pull
- Snapshot and seek (backup and restore)
- Full, granular IAM
- Pub/Sub APIs: HTTP and gRPC service, Client libraries in all 7 languages

Google Cloud

So what are some of the API features of Pub/Sub? When you're interacting with Pub/Sub, you're using a client API. That client API can do a push, can do a pull, pull subscribe, push subscribe. It can also do a streaming pull - it continues to get messages as you go along.

We can also do a snapshot and seek, so you can say: take this topic, snapshot it, go back, and look at messages that were delivered 10 minutes ago, 20 minutes ago, two days ago, etc. So this is very useful if you want to do backups. It’s also very useful if you want to do restores or playbacks. But remember that all of these are limited to about 7 days. Pub/Sub by itself is not a storage system, it's a messaging system. It gives you a snapshot capability, it gives you seek capability, but only for a certain amount of time. If you want to save your messages for longer than the seven days, store them in a real storage system like BigQuery or Cloud Storage.

Pub/Sub itself has its own IAM roles and permissions which are very, very granular. You can say who is allowed to read a message, write the message, create a topic, create a subscription, pull messages from a subscription. So very granular and you have separate permissions for each of these things.

When you're interacting with Pub/Sub, you interact with them typically through a client library. On Google Cloud there are seven supported languages, including Python, Go, C++, Java, etc. Those are all your client libraries that are available. But all those client libraries ultimately talk HTTP or gRPC and those are the two networking stacks through which the client libraries work.

Proprietary + Confidential

Service features

Fast: <500 ms 99.9% of publish, <1 s 99.9% end-to-end

Highly available: 99.999+% publish

Scalable: From 1 KB/s to 100 GB/s, with consistent performance

Durable: 3x3 sync data replication before ACK, for 7 days

Secure: Encryption in transit and rest, private network

Single, global service: Move data from anywhere to anywhere

Google Cloud

There is an SLA for the service. It's very fast. 99.9% of published messages are published within 500 milliseconds. And when you take it end-to-end, from publishing to consumption by subscription, 99.9% of messages have a latency of less than one second. But again, there's no guaranteed latency. What you're saying is that 99.9% of messages are within a second. But there's always gonna be that latency with some message that takes a long time. That's one of the things about a distributed system and something that you have to plan around.

It's highly available, 99.999 plus percent of messages will get published. So if you've tried to do a publish, you will reach Pub/Sub and you'll be able to publish it with that certainty.

It's extremely scalable. You can publish messages from a really slow and leisurely 1 kilobyte per second to extremely fast hundred gigabytes per second and you get consistent performance.

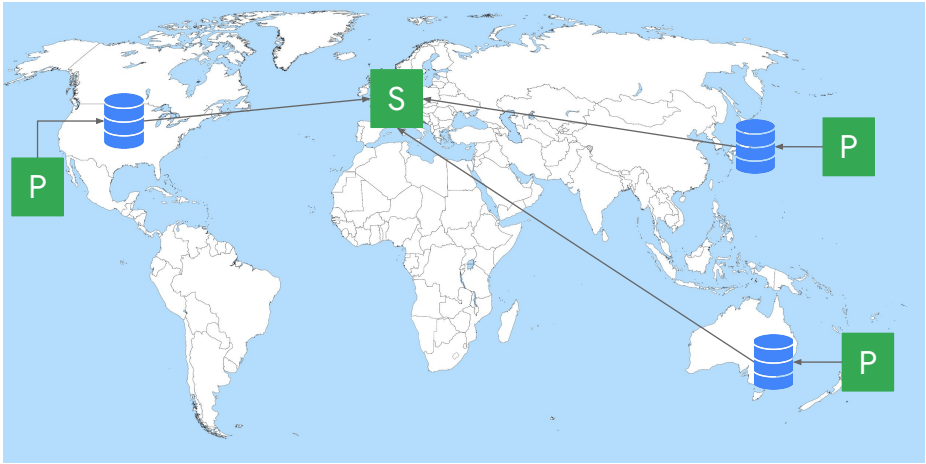
And again, messages are not lost. There is data replication. It's stored up to seven days, so even if the subscriber creates a subscription and doesn't show up for three days, all the messages are saved and available when they come back for them to process.

As with everything on Google Cloud, all your messages are encrypted in transit, it's encrypted at rest, and once it reaches Pub/Sub, it's all on a private network.

And Pub/Sub is a global service. You can move data from anywhere to anywhere in Pub/Sub. It's a single service and that's what makes it extremely good for use cases like clickstream analytics, IoT, etc.

Proprietary + Confidential

Global service

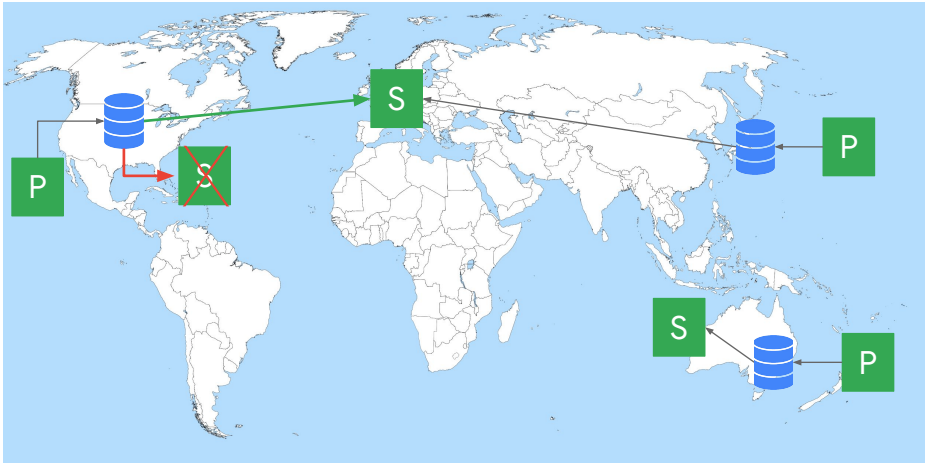


Google Cloud

So as we said, Pub/Sub is a global service. So in this case, we're showing an example of a publisher publishing to Pub/Sub in North America and a subscriber in Europe subscribing to three topics - one which is in North America, one which is in Asia, and the other one in Australia. And this all just works. You can have subscribers and publishers spread throughout the world. It's a global service and you can move data from anywhere to anywhere.

Proprietary + Confidential

Global service with regional delivery and failover



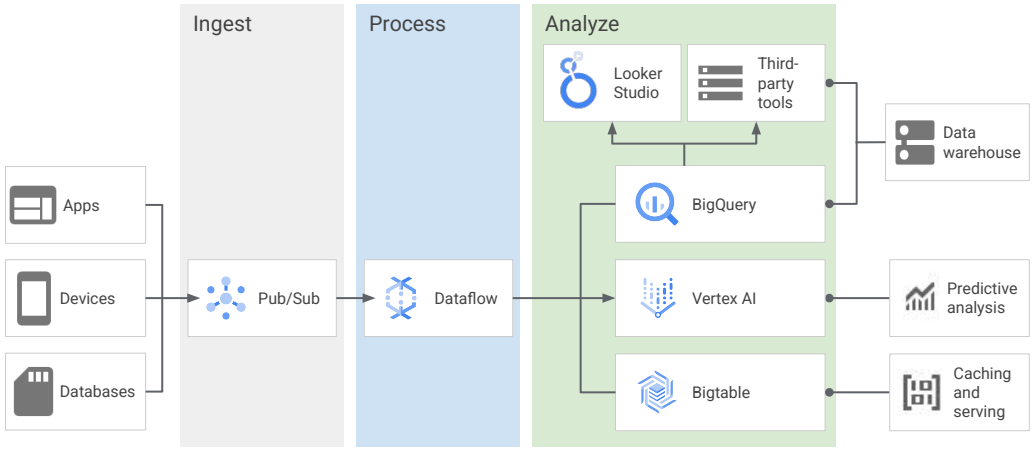
Google Cloud

This also allows you to provide regional delivery and failover. You can say that you want to make sure that all my messages get delivered to the regional client first, but if regional client is down, then send it to the client in a different region. So that's also possible.

While Pub/Sub is a global service, location can also be restricted, which can be an important feature for many types of businesses.

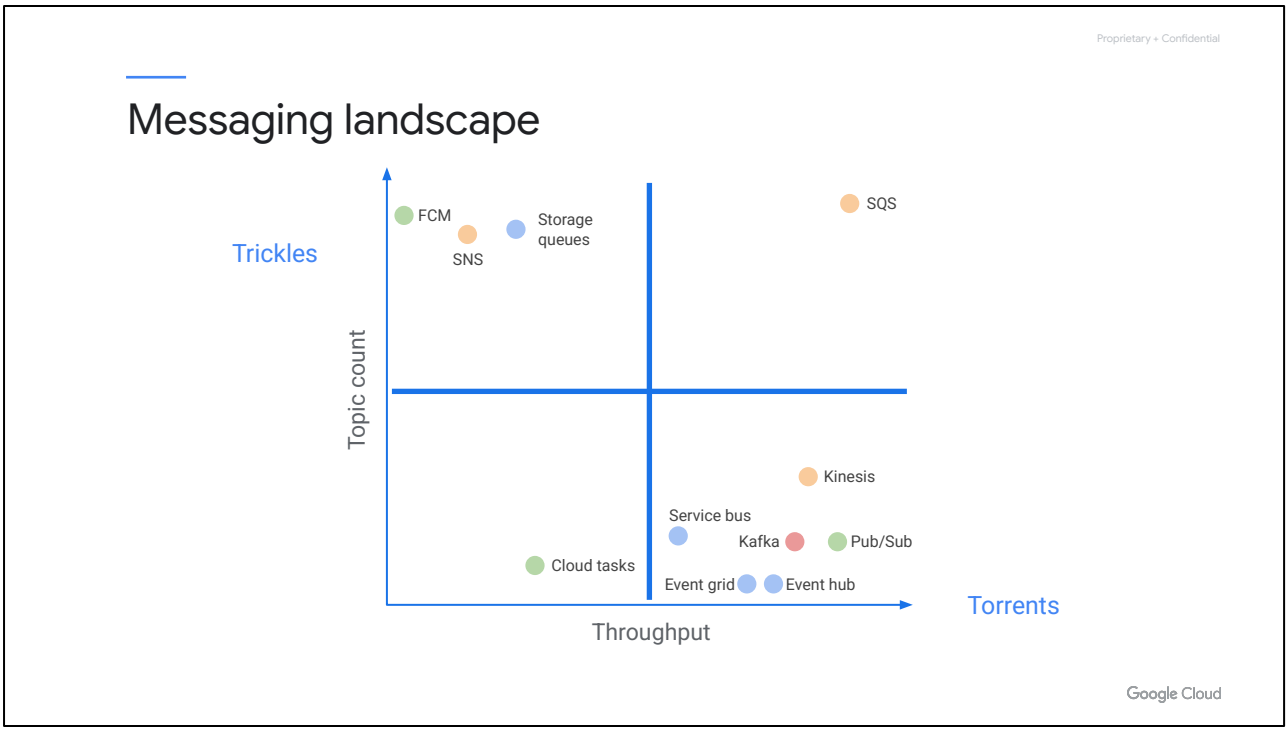
Proprietary + Confidential

Google Cloud stream analytics make event streams accessible and useful at scale



Google Cloud

Pub/Sub is a way to ingest events and all these apps are basically publishing those events into Pub/Sub. The subscription and the processing of those events in stream analytics is done with Dataflow. As we said, think of Pub/Sub and Dataflow as a combined solution because that's what gives you deduplication and gives your ordering.



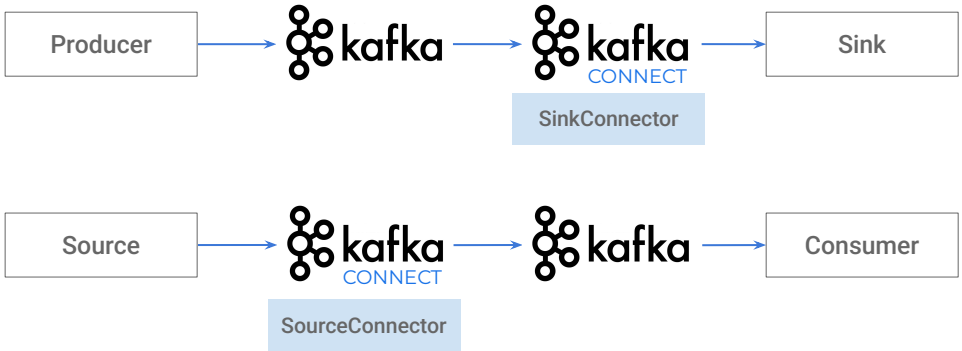
But let's go back and look at Pub/Sub itself. Where does it fit in compared to all the other messaging tools that you may be familiar with? And when you think about messaging tools, there's two axes that you need to think about. On the y-axis is the topic count. How many topics are you allowed to create in that messaging system? And on the x-axis is the throughput. How much data can you push through that system?

Pub/Sub is located in the bottom-right quadrant. Pub/Sub is for a medium number of topics - not millions of topics, but a moderate number of topics, but extremely high throughput. So that's basically where Pub/Sub plays and it's a tradeoff. If you want to support lots and lots of topics, if you want to build a text messaging system, you wouldn't do that in Pub/Sub because you would have to have a separate topic for every user and that doesn't really work.

But Pub/Sub really works for automated applications where you basically take your applications and the kinds of events that they're publishing and what they're processing. If you look on the very right hand side, you would see SQS, the simple queue service, which seems to offer extremely high throughput and a lot of topics, but it doesn't have fanout. And if you look at SNS at the top-left, which gives you lots of topics although not as much throughput, it gives you fanout, but it has way no way to reach end users. So everybody has to make a tradeoff in terms of what you're going to support. The tradeoff that Pub/Sub makes is that you have a limit on the number of topics. You don't use Pub/Sub when you need millions of topics, you use Pub/Sub when you need hundreds to thousands of topics.

Proprietary + Confidential

Kafka Connect comes in two flavors: "Sink" and "Source"

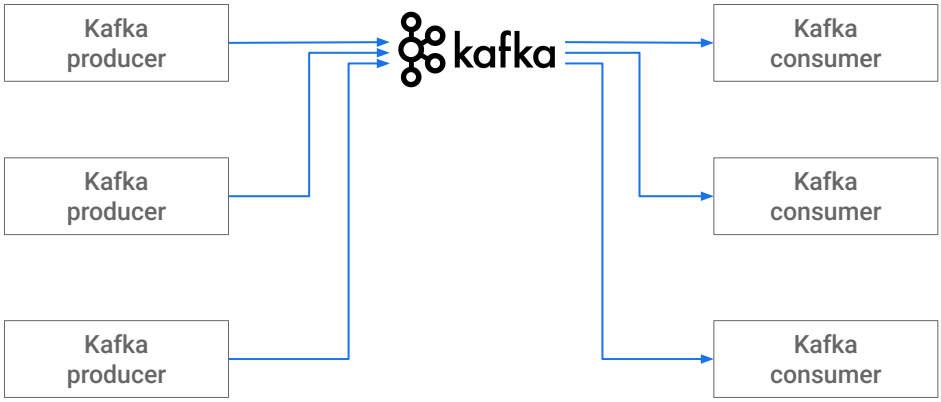


Google Cloud

So Kafka Connector comes in two flavors. There is a sink and there is a source. For example, if you have a producer publishing to Kafka, you can take that and add a Kafka Connector to the sink connector. Then you get a sink - a place in which all the messages land up. If you have a source that needs to connect to Kafka, it can connect to Kafka using the source connector and then it will be able to publish into that Kafka thing that a consumer is already set up to receive messages from.

Proprietary + Confidential

Kafka to Pub/Sub migration (1/4)

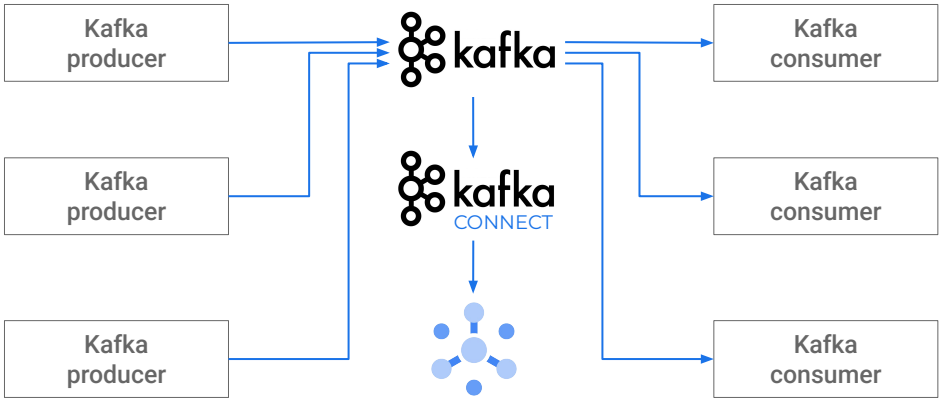


Google Cloud

So if you need to do a migration from Kafka to Pub/Sub, what do you do? So this is what the system probably looks like. You have a bunch of Kafka producers publishing into Kafka and a bunch of Kafka consumers consuming from Kafka. So if we want to replace the use of Kafka by Pub/Sub, we take advantage of the Kafka Connector.

Proprietary + Confidential

Kafka to Pub/Sub migration (2/4)

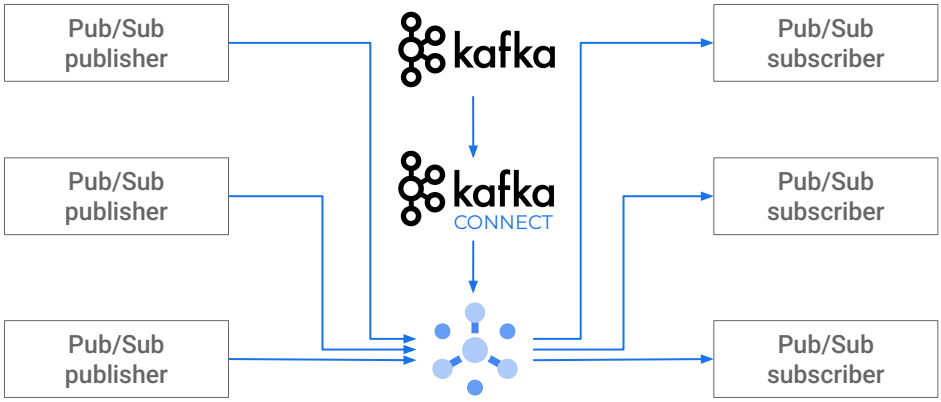


Google Cloud

We set up a Kafka connector and we put a Pub/Sub in place. Now, all the messages that are arriving in Kafka are also showing up in Pub/Sub. At that point, we go to all the Kafka consumers and switch them over so that they're not listening to Kafka, but instead they're listening to Pub/Sub. Then we go to the producers and change them as well, instead of producing to Kafka, they're now producing to Pub/Sub.

Proprietary + Confidential

Kafka to Pub/Sub migration (3/4)

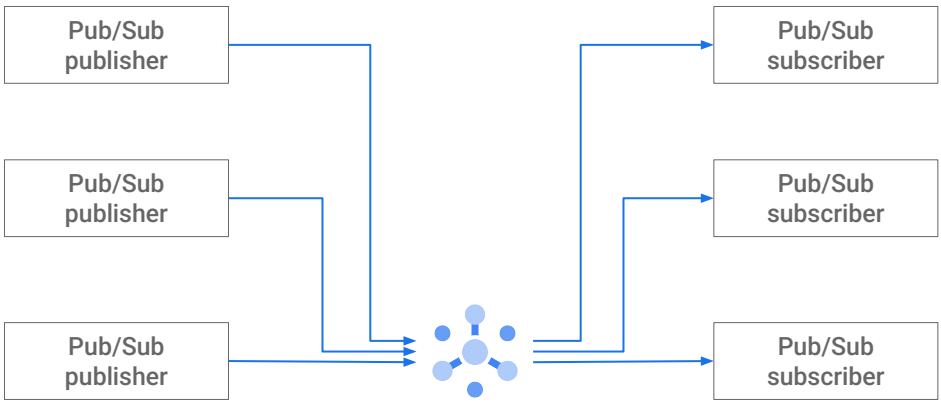


Google Cloud

So you basically have a migration strategy using a Kafka connector and then you can migrate each subscriber and producer one by one from producing or consuming from Kafka to producing or consuming from Pub/Sub. And you can do this in any order that you need.

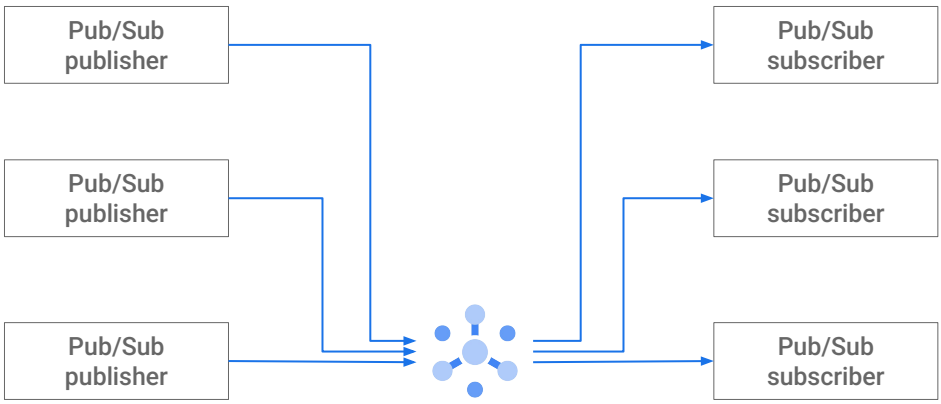
Proprietary + Confidential

Kafka to Pub/Sub migration (4/4)



Proprietary + Confidential

Pub/Sub to Kafka migration (1/4)

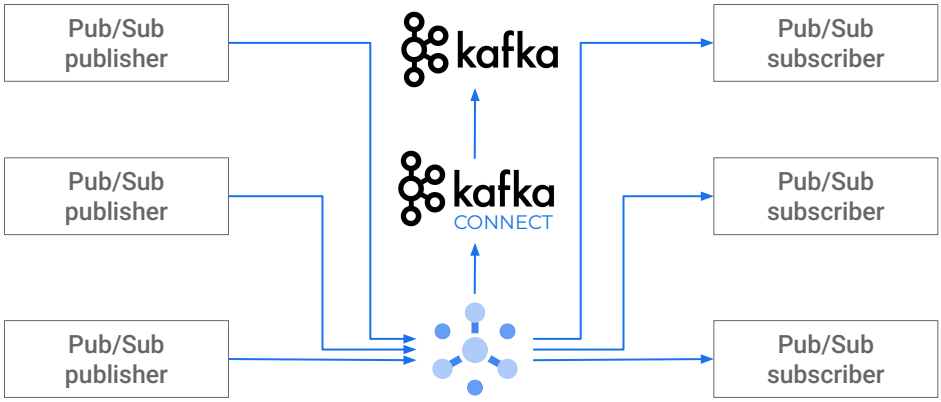


Google Cloud

What about the other way? What if you have Pub/Sub and and you want to go the other way? You want to migrate from Pub/Sub to go to Kafka - are you locked in? Not really.
If you want to go from Pub/Sub to Kafka we're just gonna go through this process in reverse. So this is what it looks like. You have Pub/Sub publishers publishing into Pub/Sub, Pub/Sub subscribers reading from Pub/Sub.

Proprietary + Confidential

Pub/Sub to Kafka migration (2/4)

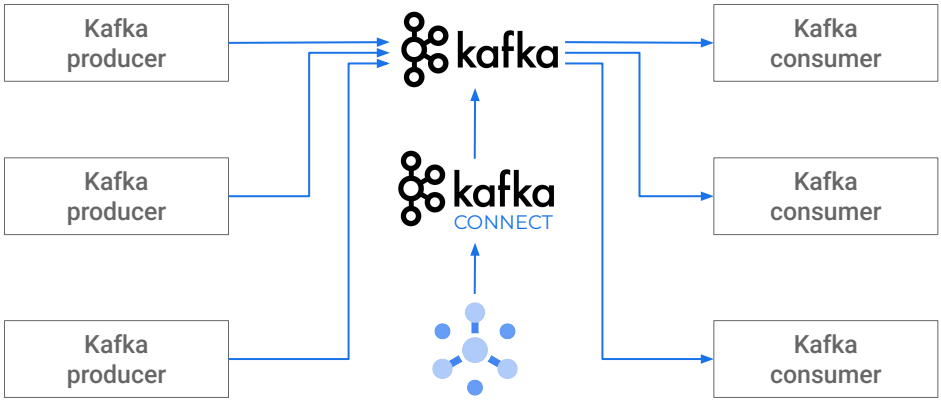


Google Cloud

You set up a Kafka Connect. Now everything in Pub/Sub is going into Kafka and you do the reverse thing.

Proprietary + Confidential

Pub/Sub to Kafka migration (3/4)

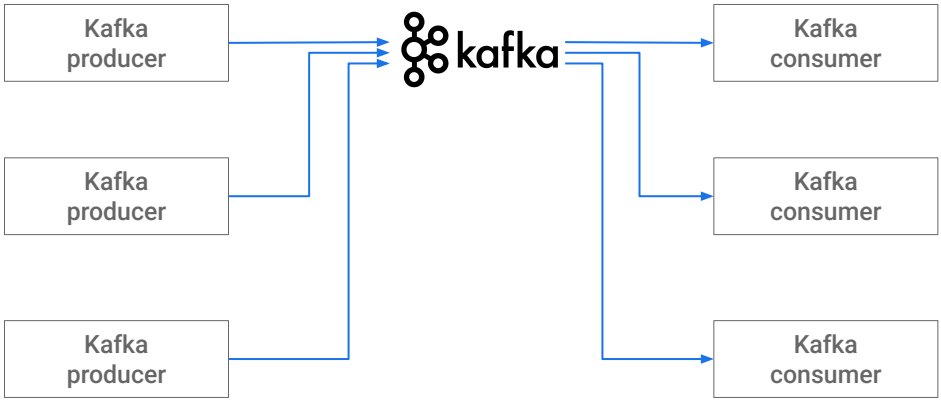


Google Cloud

You basically start converting the Pub/Sub things to use Kafka instead.

Proprietary + Confidential

Pub/Sub to Kafka migration (4/4)

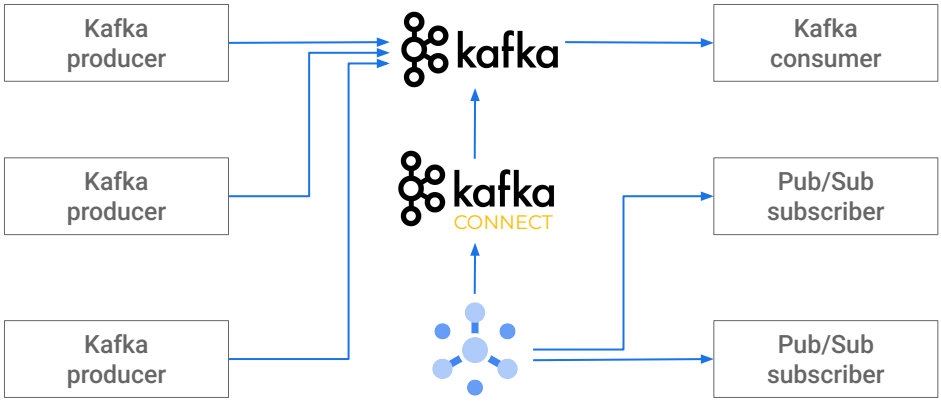


Google Cloud

So it's just a client library change at that point and you're pretty much done.

Proprietary + Confidential

Kafka and Pub/Sub hybrid



Google Cloud

But what if you want a hybrid system? Well, remember, the hybrid system was just the intermediate step where you had some systems using Kafka and some systems using Pub/Sub.
If you need a hybrid system, leave that Kafka connected place and you're fine.
When you've created Pub/Sub and you're getting the events, you probably going to look at how to improve performance. The way that you configure Pub/Sub controls the performance.

Proprietary + Confidential

There are trade offs on both sides

Publisher	
Cost:	"I'm willing to spend $x to process published messages."
Throughput:	"I need to handle y messages (bytes) per second from clients."
Latency:	"I need messages to be delivered as quickly as possible."
In all cases:	"Don't waste resources, don't overload machines." "I'm willing to introduce a delay of up to z milliseconds in order to max throughput when publishers are under load."

Subscriber	
Cost:	"I'm willing to spend $x to process messages."
Throughput:	"The topic receives y messages per second." "My downstream dependencies can only handle z messages per second."
Latency:	"Messages take w seconds to process."

Google Cloud

There are things that you have to think about both on the publisher's side and on the subscriber side. Things like cost, throughput, and latency. Typically a customer is going to come back and say, I'm going to spend this amount of money to process my published messages, or I need to handle Y messages per second because this is how much data I'm getting. Or, I need messages to be delivered as quickly as possible. So ask them what they need - cost, throughput, or latency. There are knobs that you can change and we'll do this in the lab that follows that allows you to optimize any of these things.

The same applies on the subscriber side. How much are they willing to spend to process those messages and how many topics are they going to be processing? How many messages per second do those topics receive? What are the downstream dependencies? Is there a throttle that needs to happen on the downstream dependency? Because maybe the downstream dependency can't really handle that many messages per second. And of course, latency - I want it to be as fast as possible, or I'm not very time sensitive, I can wait a little bit longer. So figure out the cost, the throughput, and latency considerations, and based on that, you can then go ahead and optimize.

Proprietary + Confidential

Summary of both publish and subscribe optimizations

Optimizing	Depends on	With the ability set to
• Cost • Latency • Throughput	• Message size • Publisher throughput • Subscriber processing latency	• Flow control • Batching • # of subscribers • Thread count in subscriber • Simultaneous publishes

Google Cloud

- You have the ability to optimize any of these things. You can have the ability to optimize cost, optimize the latency, optimize the throughput.
- And all of these depend on the message size, the publisher throughput, and the subscriber processing latency.
- And the way you do this is that you set the things on the right hand side, the flow control, the batching, the number of subscribers, the threat count in the subscriber, and the number of simultaneous publishers. You'll get to do all of this in the lab. You get to play with it, and you can kind of see how they benefit you to set the things on the right hand side allow you to optimize cost throughput or latency.

Proprietary + Confidential

Agenda

Why Streaming Analytics?

The Pub/Sub Service

Lab: Streaming Kafka to Pub/Sub

Dataflow Windows and Triggers

Dataflow Sources and Sinks

Migration and Adoption Challenges

Optional Lab: Simulating a Data Warehouse in the Cloud Using BigQuery and Dataflow

Summary

Google Cloud

We will now complete the Streaming Kafka to Pub/Sub lab.

Lab Intro

Streaming Kafka to Pub/Sub

In this lab, we launch a Confluent Kafka instance and use it to communicate with Pub/Sub.

Proprietary + Confidential

Lab objectives

- Configure a Kafka connector to integrate with Pub/Sub
- Set up topics and subscriptions for message communication
- Perform basic testing of both Kafka and Pub/Sub services

Proprietary + Confidential

Agenda

Why Streaming Analytics?

The Pub/Sub Service

Lab: Streaming Kafka to Pub/Sub

Dataflow Windows and Triggers

Dataflow Sources and Sinks

Migration and Adoption Challenges

Optional Lab: Simulating a Data Warehouse in the Cloud Using BigQuery and Dataflow

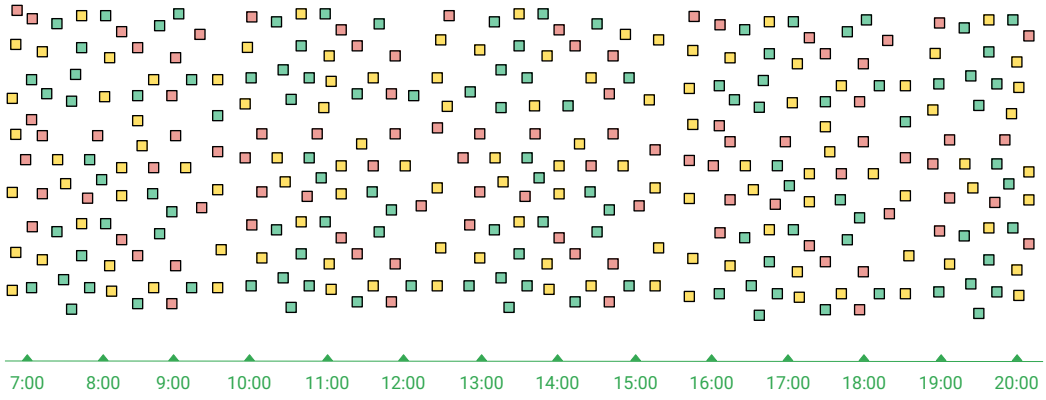
Summary

Google Cloud

The second component of a streaming analytics solution on Google Cloud is to use Dataflow. We look at Dataflow in two parts. First, we will look at windows and triggers - the way in which you process the data, and then we look at sources and sinks - ways in which we read and write the data. So first, let's look at windows and triggers.

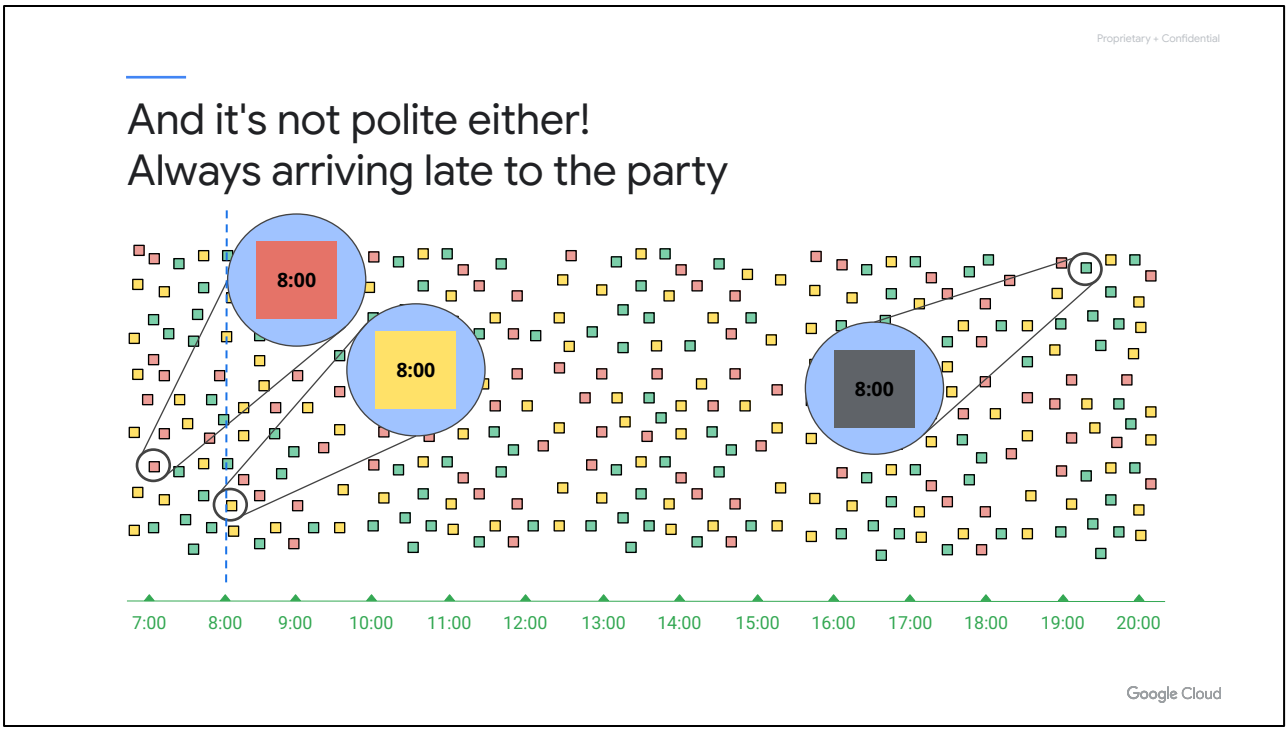
Proprietary + Confidential

Data isn't always stationary



Google Cloud

The reason that we need windows and triggers is that data is not always stationary. So imagine that you're receiving a stream of events at different times. So the bottom line is 7 o'clock, 8 o'clock, 9 o'clock, etc. Those are your times and you're basically getting a bunch of events that are coming in, or a bunch of messages that are coming in, that you need to process.

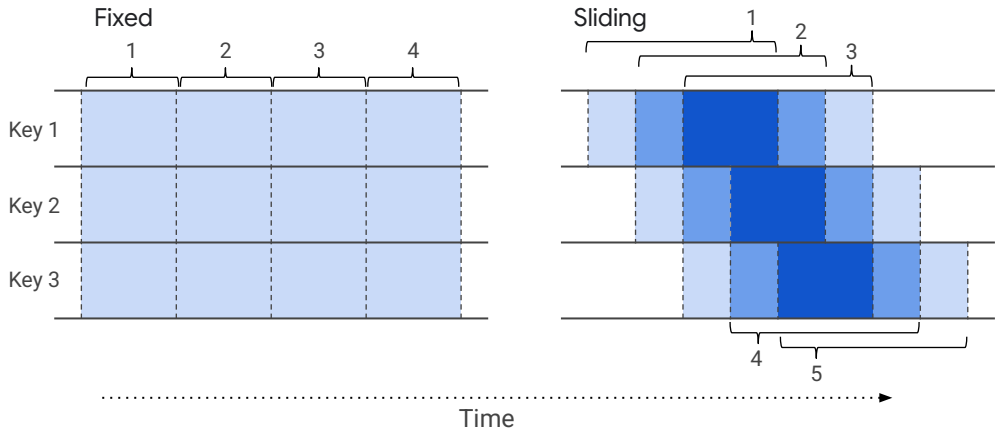


The problem is that an 8 o'clock event doesn't necessarily come in exactly at 8 o'clock. There could be some events that happen a little bit earlier, and some events that happen much later. So you have a problem of events that actually arrive on their own schedule because of network latencies or whatever the reason is. When we want to do our processing, we have to make a decision on how we carry out aggregations. If we're going to do a one hour aggregation and we start at 8 o’clock and end it at 9 o'clock, we will miss the early event and miss the later event. So we have to do something better when we define our windows to take into account things that are early, things that are late, and things that are well within the actual bar.

That's where windows come in and that’s where triggers come in because triggers would be what happens when you get a late event. It could be that you say, I'm going to start the window at 8 o'clock, I'm going to take all the early events that I've received and not throw them away, but keep them around for when the window starts, process everything from 8 to 9, maybe keep the window open a little bit longer till about 9:30, and write it out at that point. And if any event comes in later after that, we re-trigger the computations. If we're computing a maximum in the eight to nine o'clock window and we get a very late event, we update the maximum and rewrite it or ignore it. We have to decide what we're going to do about late arrivals. We have to decide how long to keep a window open in order to deal with this problem of events that come in that are out of order or are duplicates. And importantly, events that come in will have latency associated with them.

Proprietary + Confidential

Windowing divides data into event-time-based finite chunks



Google Cloud

So what windowing does is that it takes your data and divides it into event-time-based chunks. And the normal thing that you might think of would be a fixed window chunk and you get a separate window for every key. Imagine, for example, that you're trying to compute the total number of times that a customer has bought an item in a day. You're going ahead and computing the number of purchases per day and you're thinking of each day as a fixed window. The thing is, you want to compute the number of purchases by customer. The customer would be the key. So even though you're computing per day, you're actually making a lot of computations in each window, one per key. That's why you have the key there and you're doing these computations for every time window.

That's one way to do it - you may want to compute the number of purchases per day - Monday, Tuesday, Wednesday, Thursday, etc. But another way to do it is if you want to compute the number of purchases in the last hour, but don't want it computed once an hour. Instead, you want it computed once every five minutes. So that's a sliding window. You have an hour window and you're computing it every five minutes. And that's what you see on the right hand side. Again, as with the previous example, you have to do it on a key basis because different customers buy different things.

Proprietary + Confidential

Writing code

```
storeSales = p  | beam.io.ReadFromText("gs://purchases-store/*")
                | WindowInto(FixedWindows(2 * 60),
                  trigger = AfterWatermark(early = AfterProcessingTime(delay = 60), late = AfterCount(1))
onlineSales = p | beam.io.ReadFromPubSub("onlineSales")
                | WindowInto(FixedWindows(2 * 60),
                  trigger = AfterWatermark(early = AfterProcessingTime(delay = 60), late = AfterCount(1))
```

Google Cloud

When you write your code, this is what the Dataflow code looks like. Let's say, for example, we have a store sales and you're reading all of the store sales. We'll look at different sources and sinks later. In this case, we're saying I'm going to read all of the store sales and I'm going to window into it with fixed windows. In other words, I'm going to do a fixed window where there's no overlap. The size of the window in this case is 2 x 60, which is 120 minutes. So in two minute windows, I'm going to compute whatever it is I'm going to compute, but I'm also going to add a trigger.

The trigger is that we're going to take into account a delay of 60 seconds every time. So late equals AfterCount (1), meaning every time I receive a late arrival event, I'll go back and I'll update whatever statistics I'm computing. So in other words, what that trigger is saying is that we're going to do it after a watermark.

What's a watermark? A watermark is something that Dataflow keeps in mind. It's looking at all the events that are coming in and it's saying, well, on average, all these events are late because of network traffic, etc., by 30 seconds. So your two minute window from 1:01 to 1:03, I'm going to keep it open for 30 extra seconds. That's what the watermark is. The watermark is something that Dataflow calculates based on the latency of messages that are coming in. And you're saying that beyond that watermark, whenever there's a late arrival, please also go ahead and re-trigger and recompute.

So that's what the first part says. So compute the store sales every 2 minutes, making sure to update whatever count you're doing whenever there's a late arriving record.

Then you say, I'm going to read one more thing - online sales, which are the exact same thing. Now you have two sets of what’s called PCollections in Dataflow that you can then go ahead and do other processing with. So that's basically how you set up a window or you set up a trigger in Dataflow.

Proprietary + Confidential

Agenda

Why Streaming Analytics?

The Pub/Sub Service

Lab: Streaming Kafka to Pub/Sub

Dataflow Windows and Triggers

Dataflow Sources and Sinks

Migration and Adoption Challenges

Optional Lab: Simulating a Data Warehouse in the Cloud Using BigQuery and Dataflow

Summary

Google Cloud

In the previous lesson when I showed you the code, you saw that we were reading from Cloud Storage in order to set up a window and trigger that we are reading our store sales and online sales. What else can you read from?

Proprietary + Confidential

Pub/Sub: Python

```
pCollection pc = p | beam.io.ReadFromPubSub("inputTopic")

                                                    ...
pc | beam.io.WriteToPubSub("storeSales")
```

Google Cloud

Well, you can read from Pub/Sub. This is how you would read from Pub/Sub - you'd specify the name of the input topic and you would end up with a PCollection. And if you want to publish to Pub/Sub, Beame can do that as well. Apache Beam is the API that you use and Dataflow is the executor service. So in this case, we're basically writing to Pub/Sub and we're writing to the topic name storeSales.

Proprietary + Confidential

Pub/Sub: Java

```
PCollection<String> pc = p.apply(
    PubSubIO.readStrings("topic"));

PCollection<Type> pc = p.apply(
    PubSubIO.readAvrosWithBeamSchema("topic"));

...

pc.apply(PubSubIO.writeStrings("topic"));
```

Google Cloud

You could do that in Python as I showed on the previous slide, and you can also do it in Java. This is showing you how to do it in Java. You're going ahead on the pipeline and saying apply *PubSubIO.readStrings* passing in the topic name. And similarly, you can read Avro files with *readAvro* with some kind of a beam schema and you can write strings, you can write Avro profiles, you can write JSON, you can go ahead and choose your message and write them.

So you can read and write from Pub/Sub either in Python or in Java.

Proprietary + Confidential

BigQuery: Python

```
Table_spec = bigquery.TableReference(project, dataset, table)
pCollection pc = p | beam.io.Read(beam.io.BigQuerySource(table_spec))

                                    ...

pc | beam.io.WriteToBigQuery(
   table_spec,
   write_disposition=beam.io.BigQueryDisposition.WRITE_TRUNCATE,
   create_disposition=beam.io.BigQueryDisposition.CREATE_IF_NEEDED)
```

Google Cloud

If you want to read from BigQuery in Beame, you use a *beam.io.BigQuerySource* passing in the name of the table. And if you want to write to BigQuery, you pass in the *table_spec*, very similarly pass in the table, but then you also tell it what should happen if the table doesn't exist. So that's where you see CREATE_IF_NEEDED. So in other words, if the table doesn't exist, we want this beam pipeline to create the table. But what happens if the table already exists? In this case, we are saying truncate the table and write. In other words, replace the content of the table. The other option would be to append to the table.

So you have the option of truncating or appending and you have the option of creating if needed, or only writing to an already existing table. So when you write, you basically specify a right disposition and you specify a create disposition. When you read, you specify the name of a table. It's also possible to specify a query. And then the first thing that will happen is the query will get run and the Dataflow pipeline will basically start from the temporary table that's created from the query result.

Proprietary + Confidential

BigQuery: Java

```
PCollection<TableRow> pc = p.apply(BigQueryIO.readTableRows.from(tableSpec));


PCollection<TableRow> pc = p.apply(BigQueryIO.readTableRows.from(tableSpec)
       .fromQuery("SELECT …..."));


                                        …


pc.apply(BigQueryIO.write("table")
    .useBeamSchema()
    .optimizeWrites());
```

Google Cloud

You can do the same thing from Java as well. Here is *BigQueryIO.readTableRows* from a particular table. And you can also do it as a set from a query where you're doing *.fromQuery* passing in the query. And when you write, you do a write and as before, you can set a read and write disposition, although we're not showing that in Java because the lines get very long.

Proprietary + Confidential

BigQuery: Dynamic destinations

```
pc.apply(BigQueryIO.<Purchase>write(tableSpec)
    .useBeamSchema()
   .to((ValueInSingleWindow<Purchase> purchase) -> {
        return new TableDestination("project:dataset-" +
           purchase.getValue().getUser() + ":purchases", "");
    });
```

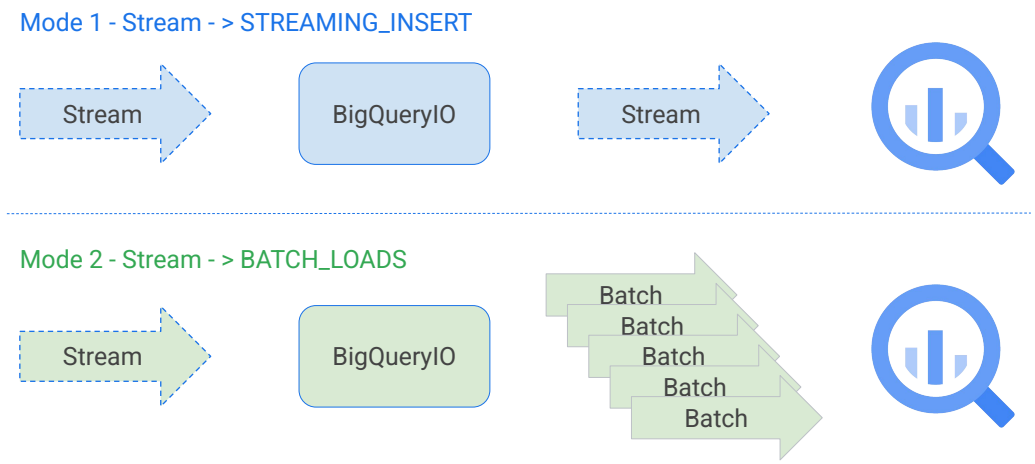
Google Cloud

One of the cool things in BigQuery that's relatively new as far as a BigQuery and Beame integration, is that you can look at the record and decide which table to write to. That's called a dynamic destination and this is how you do it.

You're saying, I'm going to take my purchase data and write it to a table destination where I will get the user name of the purchase. You're basically writing to that particular dataset. So you're choosing the table name or the dataset name based on the content of what you want to write. And this is how you do that - you create a table destination dynamically.

Proprietary + Confidential

Detail BigQueryIO: Sink performance considerations



Google Cloud

When you're writing to BigQuery, there are two ways to write to it.

1. One way is what's called a streaming insert. As the data comes in, use the *BigQueryIO* class that we just showed you to stream it into BigQuery.

2. The second option is as the data comes in, use the *BigQueryIO* class, but tell it to combine a bunch of records and do micro batching and writing into BigQuery. So in the options that you're writing, how do you choose between streaming insert and batch loads? When do you choose what?

Proprietary + Confidential

BigQuery: Streaming writes

Streaming inserts	Batch loads
`withMethod(STREAMING_INSERTS)`	`withMethod(BATCH_LOADS)` or `withTriggeringFrequency(..)`
• Optimized for low latency • 100K QPS quota per table. (BigQuery team can provide more) • Pay for every insert • Might sometimes duplicate data	• Write output to files and periodically load into BigQuery • Free! • BigQuery has daily quotas (don't load too often - every 10 min is usually safe) • No duplicates • Requires more tuning to get right

Google Cloud

- For streaming inserts, the way you do that is *withMethod(streaming inserts)* and it's really optimized for extremely low latency.
- Because it's expensive, there's a soft quota of 100k queries per second per table. But if you need to write more, just reach out to the team and we'll basically increase it. This is meant more to protect you than than anything else.
- When you do a streaming insert, you're actually paying for every insert.
- And because Dataflow doesn't quite know about whether the insert succeeded or not, as with Pub/Sub, there might be sometimes duplicate data.

- On the other hand, for batch loads you use *withMethod(batch_loads)* and you specify a triggering frequency. For example, I want to write it every five minutes, or I want to write it every minute. And what will happen is that Dataflow will accumulate all these records over that time and then write them out.
- This is free - batch loads of free.
- However, there is a daily quota, so you don't want to do it every minute. You don't want to do it every five minutes. If you try to do it every 10 minutes, you're usually very safe. If you need latency lower than 10 minutes, then you should use a streaming insert.
- The good thing about a batch load is that there will be no duplicates, but you will have to do some tuning to get all the messages that you want without

- So the choices are between a streaming insert, which is what you want for high-performance applications, and batch loads, which is what you want for time-insensitive operations.

Proprietary + Confidential

Agenda

Why Streaming Analytics?

The Pub/Sub Service

Lab: Streaming Kafka to Pub/Sub

Dataflow Windows and Triggers

Dataflow Sources and Sinks

Migration and Adoption Challenges

Optional Lab: Simulating a Data Warehouse in the Cloud Using BigQuery and Dataflow

Summary

Google Cloud

We started this module by saying that streaming analytics is the next hot thing in big data analytics. But as with anything that's new, there going to be some adoption challenges. People will ask: Why should I do this?

Proprietary + Confidential

On-premises/IaaS batch-only users

Challenge

- "Batch is cheap"
- "Batch is observable"
- Batch can be done in SQL, Scala, my favorite language

Path to success

- Start with batch Google Cloud: lift-and-shift
- Try Dataflow for batch jobs
- ... or Beam on Flink & Spark runners;
- Finance back office apps
- ... in Scala, Python, SQL
- Try a version of your pipeline with a streaming IO:
  - The only batch and streaming product
  - Serverless auto-scaling and performance of the Streaming Engine

Google Cloud

If you have on-premises, batch-only users, they're basically thinking of it in terms of infrastructure as a service.

The **challenge** that you will get is that, hey, batch is cheap - I know exactly what's happening, and I can do batch in Scala, I can do it in SQL, I can do in whatever language I want. Why should I basically go learn Dataflow and Pub/Sub and learn all of these new concepts and new things to do stream. What's the point?

The **path to success** with those kinds of customers is to start with batch. Do a lift and shift. Take their batch workloads, move them over, and then do Dataflow for batch jobs. You don't have to do it in Dataflow. If you write it in Apache Beam, you can run it in Flink, you can run it in Spark, you can run it in Scala, you can run it in Python, you can do it in SQL. So you have a bunch of different options. Use the thing that's most familiar. You can write Apache Beam, you can do Beam SQL, you can do Beam Python, you can do Beam in Java, you can do Beam in Scala Scio - a library by a Google Cloud customer called Spotify. So you could do all of those things. Get them familiar with this idea that Apache Beam is an API that allows you to do batch and then switch the input and output.

Try a version of this pipeline with streaming IO. So it may be the exact same thing as a batch product, but you get serverless auto-scaling, you get the performance of the streaming engine, and you get more real-time data. And that might convince them because it's now moving from something that's familiar to something that has value add.

Proprietary + Confidential

Kafka users

Challenge

- ❤️ open source
- Kafka is cheaper
- ❤️ ordered logs
- ❤️ binaries on Compute Engine | Google Kubernetes Engine
- ... “not a Java shop”

Path to success

- Try Kafka with Dataflow: the only way to get managed stream processing
- Understand SLOs: low cost = low availability
- Switch to Pub/Sub because no partitions, global, integrated and Dataflow orders and dedupes Pub/Sub
- Dataflow supports Python, SQL (Alpha), Scala (3rd party), Go (in dev)

Google Cloud

If you have users who are already using Kafka, the challenge is that they’re going to say: I love open source and Kafka is cheaper. I love ordered logs and I can run the binaries and Compute Engine or Kubernetes Engine, and if it’s Kafka I don’t have to use Java. Why should I look into things like Dataflow, why should I look into things like Pub/Sub where ordering isn't guaranteed, and so on.

Try Kafka with Dataflow. Dataflow will give you managed stream processing and one of the IO connectors that’s supported is Kafka. So you don't have to switch to Pub/Sub.
Keep using Kafka, but understand the SLOs - low cost equals low availability. With Pub/Sub you get high availability, so that's why it's a little more expensive.
Once you have them working with Dataflow, think about where it makes sense to replace Kafka with Pub/Sub. You get a global service that's integrated and because you're already using Dataflow, you get your ordering and you get your deduplication. So that problem of ordering and duplication is no longer a concern because processing is being done in Dataflow.
Remember, Dataflow supports Python, SQL, Scala, and Go, so you have a variety of options for wherever the developments skills are to develop a Dataflow pipelines.

Proprietary + Confidential

AWS Kinesis users

Challenge

- ❤️ SQL
- ❤️ Firehose (managed connectors)
- Kinesis Streams is cheaper than Pub/Sub

Path to success

- Dataflow has SQL from 2019
- BigQuery has SQL and low latency
- Dataflow Google-provided templates ~ Firehose
- Kinesis ingestion is cheaper but Kinesis Analytics costs make up for it

Google Cloud

If you have Kinesis users, then the challenges could be: Oh, I love SQL. I love managed connectors like Firehose and Kinesis Streams is cheaper than Pub/Sub.

Well, first of all, Dataflow has equal support now, so go ahead and do it.
BigQuery has SQL, and the amazing thing is Dataflow and BigQuery support the same SQL dialect and BigQuery gives you a low latency.
Dataflow does have a template for Firehose.
And the thing that they may be missing is that Kinesis ingestion is cheaper, so yes, it's cheaper than Pub/Sub, but Kinesis Analytics is super expensive where Dataflow is not. So if you take the combination of Pub/Sub and Dataflow, we are very cost effective.

Proprietary + Confidential

Customer highlight: Spotify

Goal:

Reliable global user behavior analytics, stateless front-ends, real-time features for machine learning

Why migrate:

- Existing Kafka infrastructure highly unreliable
- Managing biggest on-prem Hadoop cluster in Europe

Migration path:

- Pub/Sub replaced Kafka and solved reliability problems
- Forklift migration of Hadoop/Spark to Dataproc
- Develop Dataflow pipelines and run 100,000s jobs/day
- Developed Scala-based wrapper around Dataflow (Scio)

Why Google won:

- Best-in-class streaming technology
- Good early experience with Pub/Sub

Google Cloud

Take Spotify as an example of a customer who successfully uses streaming analytics products.

Spotify is the music application - it streams music and has millions of users around the world who listen to music. So their **goal** is that they want reliable user behavior analytics because they send out playlists of the kind of music that users would like every day. They want the frontend to be stateless, the frontend being the mobile app. All the states should be on the on the server so that regardless of whether you're accessing Spotify on your phone or on your TV or on your desktop computer, you get the same playlist and get the same everything. But at the same time, they know that they want real-time features for doing machine learning, for doing song suggestions, figuring out what you are skipping and so on. So they need that real-time analytics capabilities in order to support global user behavior analytics.

So **why did they have to migrate**? Because they were initially using Kafka and it was very unreliable and they were managing the biggest on-prem Hadoop cluster in Europe. They moved from Kafka and Hadoop, and we'll talk about Hadoop migration later in this course, but Spotify moved from Kafka to Pub/Sub and moved from Hadoop processing - not to Dataproc necessarily, but to Dataflow.

So their **migration path** was that they used Pub/Sub to replace Kafka - that solved the reliability problems, and then initially they did a lift and shift of Hadoop and Spark workload to Dataproc. And over time they started replacing their Dataproc pipelines with Dataflow pipelines because that would allow them to run hundreds of thousands

of jobs per day and because their development expertise was primarily Scala, they love Scala, not necessarily Java, they built a Scala wrapper around the Java API. They call it Scio, it's open source and that's how much Spotify is invested in the Pub/Sub Dataflow story.

So **why did Spotify choose Google Cloud**? Because Google have the best streaming technology and because Spotify saw this early advantage of the difference in reliability between Kafka and Pub/Sub.

Proprietary + Confidential

Agenda

Why Streaming Analytics?

The Pub/Sub Service

Lab: Streaming Kafka to Pub/Sub

Dataflow Windows and Triggers

Dataflow Sources and Sinks

Migration and Adoption Challenges

Optional Lab: Simulating a Data Warehouse in the Cloud Using BigQuery and Dataflow

Summary

Lab Intro

(Optional) Simulating a Data Warehouse in the Cloud Using BigQuery and Dataflow

Proprietary + Confidential

Lab objectives

- Build a data pipeline that ingests data into BigQuery simulating a batch transformation
- Create a streaming Dataflow pipeline that ingests streaming data and updates a BigQuery table using a Dataflow template

Proprietary + Confidential

Agenda

Why Streaming Analytics?

The Pub/Sub Service

Lab: Streaming Kafka to Pub/Sub

Dataflow Windows and Triggers

Dataflow Sources and Sinks

Migration and Adoption Challenges

Optional Lab: Simulating a Data Warehouse in the Cloud Using BigQuery and Dataflow

Summary

Google Cloud

That concludes the ‘Streaming Analytics’ module. Let me remind you of what you learned.

Proprietary + Confidential

Summary

Stream analytics has many applications within areas such as data integration and online decision-making.

Google Cloud stream analytics make event streams accessible and useful at scale.

Google Cloud

Stream analytics is then next hot thing in big data processing. There are many applications in areas such as data integration and particularly in online decision making.

And the way you do streaming analytics on Google Cloud is that you combine Pub/Sub and Dataflow, stream it into BigQuery, and make it available for things like machine learning and data analytics.

On Google Cloud, stream analytics make event streams accessible and useful at scale. So whenever you talk about data warehousing on Google Cloud, you want to consider whether pipelines that today are done in batch could be modernized and made streaming instead. Because it's very easy to convert a batch Dataflow pipeline to be a streaming Dataflow one and make the event stream accessible in real time and real-time data has a lot more value than data that is even just a day old.